



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Representación de la información

Números enteros

Sistemas Digitales

2do Cuatrimestre 2026

FCEN - UBA

Representación de números enteros

El concepto

lo que **queremos**
representar:
un número, un color, un
carácter...

“el número 25”

Para un mismo concepto existen múltiples
representaciones posibles:

El concepto

lo que **queremos**
representar:
un número, un color, un
carácter...

“el número 25”

Para un mismo concepto existen múltiples
representaciones posibles:

00011001 — sin signo

El concepto

lo que **queremos**
representar:
un número, un color, un
carácter...

“el número 25”

Para un mismo concepto existen múltiples
representaciones posibles:

00011001 — sin signo

00011001 — complemento a 2

El concepto

lo que **queremos**
representar:
un número, un color, un
carácter...

“el número 25”

Para un mismo concepto existen múltiples
representaciones posibles:

00011001 — sin signo

00011001 — complemento a 2

10011001 — exceso 128

El concepto

lo que **queremos**
representar:
un número, un color, un
carácter...

“el número 25”

Para un mismo concepto existen múltiples
representaciones posibles:

00011001 — sin signo

00011001 — complemento a 2

10011001 — exceso 128

La elección de representación es una **convención**: hay que acordarla de antemano para poder interpretar los bits correctamente. A lo largo de esta clase vamos a ver varias convenciones posibles para representar enteros.

Vector de bits:

$$X = (x_{w-1}, x_{w-2}, \dots, x_0)$$

$$x_i \in \{0, 1\}$$

Interpretación como “sin signo”:

$$\text{B2U}_w(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

Observaciones

- No hay bit de signo.
- Rango representable:

$$0 \leq \text{B2U}_w(X) \leq 2^w - 1$$

- **Sin signo**
representa únicamente números positivos

- **Sin signo**
representa únicamente números positivos
- **Con signo**

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**
se usa el primer dígito (bit) para indicar el signo de la magnitud

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**
se usa el primer dígito (bit) para indicar el signo de la magnitud
 - **Exceso m**
Represento n como $m + n$. Los valores a izquierda de m serán interpretados como negativos

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**
se usa el primer dígito (bit) para indicar el signo de la magnitud
 - **Exceso m**
Represento n como $m + n$. Los valores a izquierda de m serán interpretados como negativos
 - **Complemento a 2**

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**
se usa el primer dígito (bit) para indicar el signo de la magnitud
 - **Exceso m**
Represento n como $m + n$. Los valores a izquierda de m serán interpretados como negativos
 - **Complemento a 2** ← **Esta es la más utilizada de las representaciones con signo**

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**
se usa el primer dígito (bit) para indicar el signo de la magnitud
 - **Exceso m**
Represento n como $m + n$. Los valores a izquierda de m serán interpretados como negativos
 - **Complemento a 2** ← **Esta es la más utilizada de las representaciones con signo**

En todos los casos, lo representable dependerá de la **cantidad de bits** que utilicemos para la representación, por eso es importante que lo digamos.

Veamos ejemplos con rangos de representación para datos de 3 bits. El rango es:

v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7
-------	-------	-------	-------	-------	-------	-------	-------

Los numerales son los mismos en todos los casos, lo que va a cambiar van a ser las magnitudes asociadas a cada elemento (cómo los interpretamos). Los numerales son:

v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7
000	001	010	011	100	101	110	111

Las magnitudes asociadas al cada numeral son:

Posición	v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7
Dato	000	001	010	011	100	101	110	111
Sin signo	0	1	2	3	4	5	6	7
Signo+magnitud	0	1	2	3	-0	-1	-2	-3
Exceso $m(m=2)$	-2	-1	0	1	2	3	4	5

Noten que para signo+magnitud hay dos representaciones asociados al cero

Motivación

- Es el método más utilizado para representar enteros con signo en computadoras.
- Su principal ventaja es que las operaciones aritméticas, como **la resta**, se simplifican enormemente.
- La resta de dos números ($A - B$) puede realizarse como la suma de A más el inverso aditivo de B.
- Existe una única representación para el cero.

$$\text{C2}(b) = 2^n - b$$

Si sumamos a y $\text{C2}(b)$:

$$a + \text{C2}(b) = a + (2^n - b) = a - b + 2^n$$

Como trabajamos en aritmética de n bits, los cálculos se hacen módulo 2^n :

$$a - b + 2^n \equiv a - b \pmod{2^n}$$

Por lo tanto, tenemos que:

$$a - b \equiv a + \text{C2}(b) \pmod{2^n}$$

Vector de bits:

$$X = (x_{w-1}, x_{w-2}, \dots, x_0)$$

$$x_i \in \{0, 1\}$$

En complemento a 2:

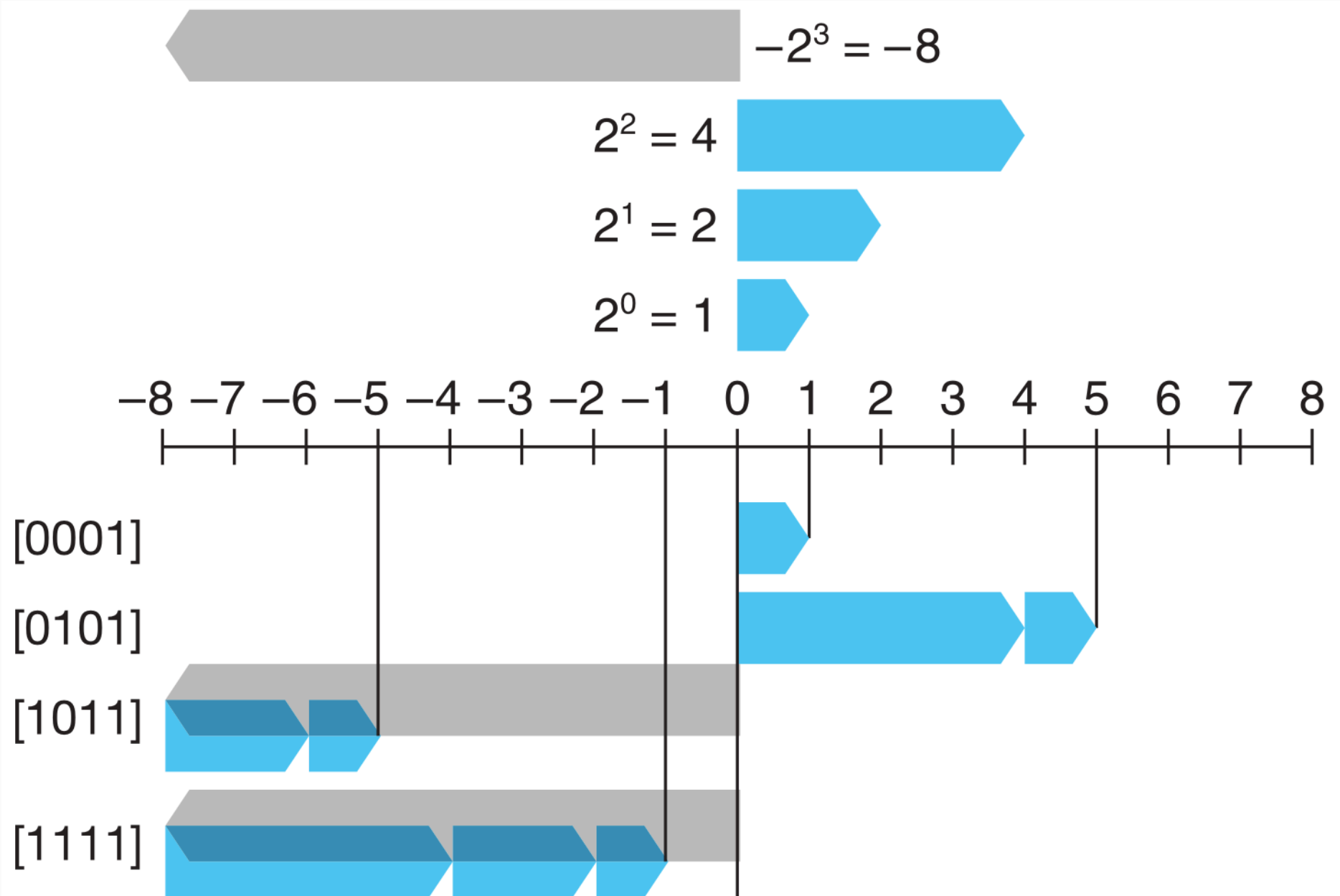
$$\text{B2T}_w(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

Observaciones

- El bit más significativo x_{w-1} nos va a indicar si es positivo o negativo (nos determina el *signo*).
- Rango representable:

$$-2^{w-1} \leq \text{B2T}_w(X) \leq 2^{w-1} - 1$$

Enteros con signo: Complemento a 2



No signado

Complemento a 2

$$\text{B2U}_w(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i \quad \text{B2T}_w(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

	Decimal	Binario	Hexa
x	121	01111001	0x79
y	-121	10000111	0x87

Bit de signo

En complemento a dos, el bit más significativo (MSB) indica el signo: 0 para positivo, 1 para negativo.

$x = 121 : \text{B2U}(01111001)$
 $x = -121 : \text{B2T}(10000111)$

Peso (2^i)	121		-121	
1	1	1	1	1
2	0	0	1	2
4	0	0	1	4
8	1	8	0	0
16	1	16	0	0
32	1	32	0	0
64	1	64	0	0
± 128	0	0	1	-128
Suma:	121		-121	

Ahora: misma cadena binaria, pero distinta interpretación

x = 150 : B2U(10010110)
x = -106 : B2T(10010110)

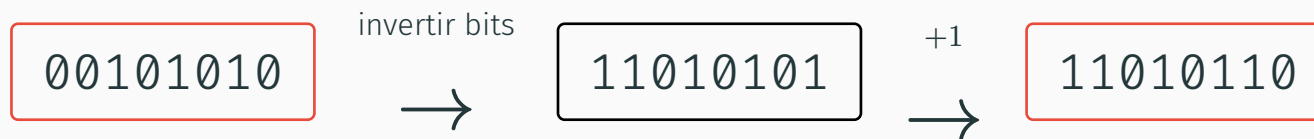
Peso (2^i)	150		-106	
1	0	0	0	0
2	1	2	1	2
4	1	4	1	4
8	0	0	0	0
16	1	16	1	16
32	0	0	0	0
64	0	0	0	0
±128	1	128	1	-128
Suma:	150		-106	

Truco

Para codificar un número en complemento a 2:

- Codificar el número como unsigned
- invertir los bits (NOT) y sumar 1.

Codificar -42 en CA2 de 8 bits

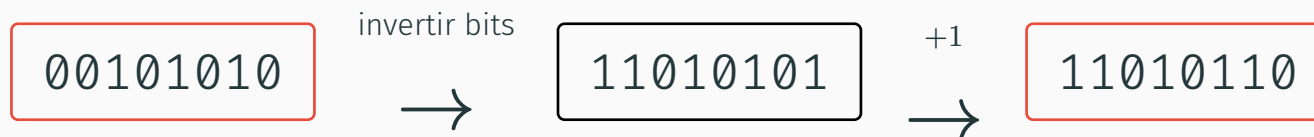


Truco

Para codificar un número en complemento a 2:

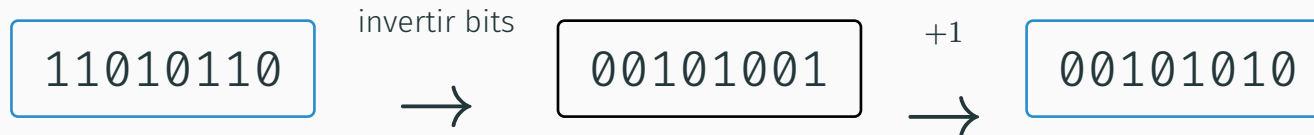
- Codificar el número como unsigned
- invertir los bits (NOT) y sumar 1.

Codificar -42 en CA2 de 8 bits



$\downarrow$ el mismo algoritmo, aplicado de nuevo $\downarrow$

Decodificar de vuelta a decimal



el procedimiento (invertir + 1) obtiene el inverso aditivo. El valor positivo suele ser más fácil de interpretar.

Límites: no signados

$$U_{\text{Min}} = 0 \quad (00\dots 0)$$

$$U_{\text{Max}} = 2^w - 1(11\dots 1)$$

Límites: complemento a dos

$$T_{\text{Min}} = -2^{w-1} \quad (10\dots 0)$$

$$T_{\text{Max}} = 2^{w-1} - 1(01\dots 1)$$

Valores para $w = 16$

	Decimal	Hexa	Binario
U _{Max}	65535	FF FF	11111111 11111111
T _{Max}	32767	7F FF	01111111 11111111
T _{Min}	-32768	80 00	10000000 00000000
-1	-1	FF FF	11111111 11111111
0	0	00 00	00000000 00000000

	4	8	16	32
UMax	15	255	65535	4294967295
TMax	7	127	32767	2147483647
TMin	-8	-128	-32768	-2147483648

Observaciones

- $|TMin| = TMax + 1$ (rango asimétrico)
- $UMax = 2 \cdot TMax + 1$

X	B2U(X)	B2T(X)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

- **Equivalencia**

- Mismo encoding para valores no negativos

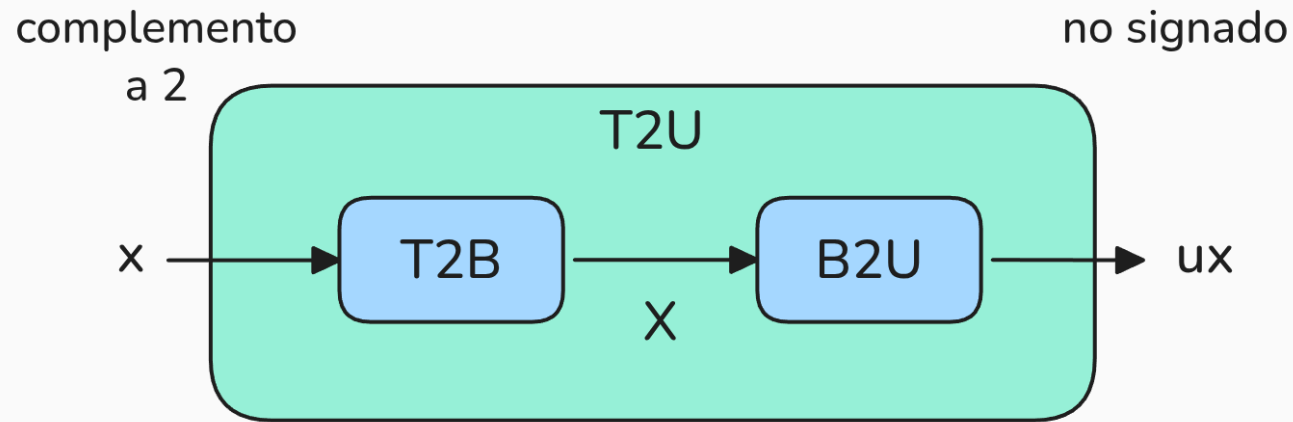
- **Unicidad**

- Cada patrón representa un único entero
- A cada entero representable le corresponde un único patrón

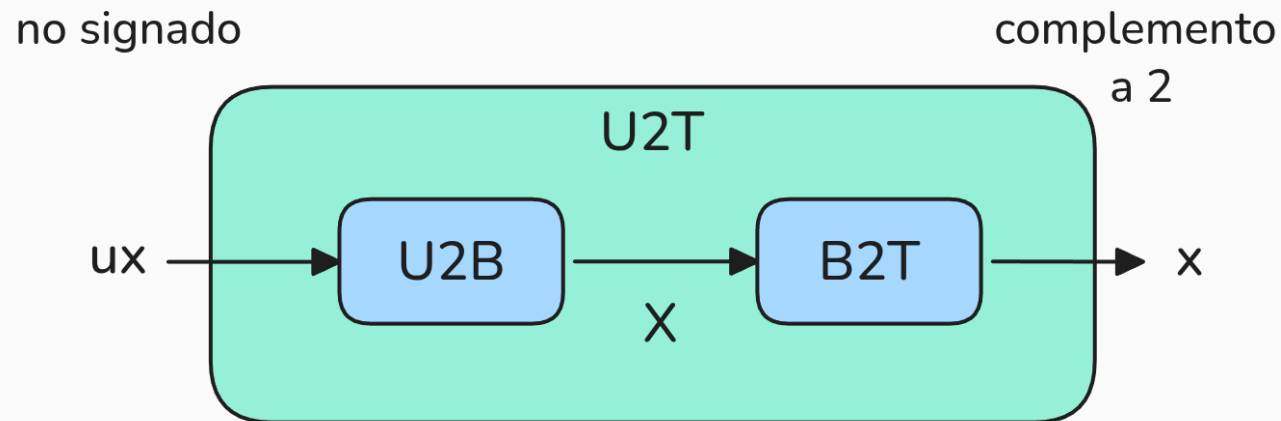
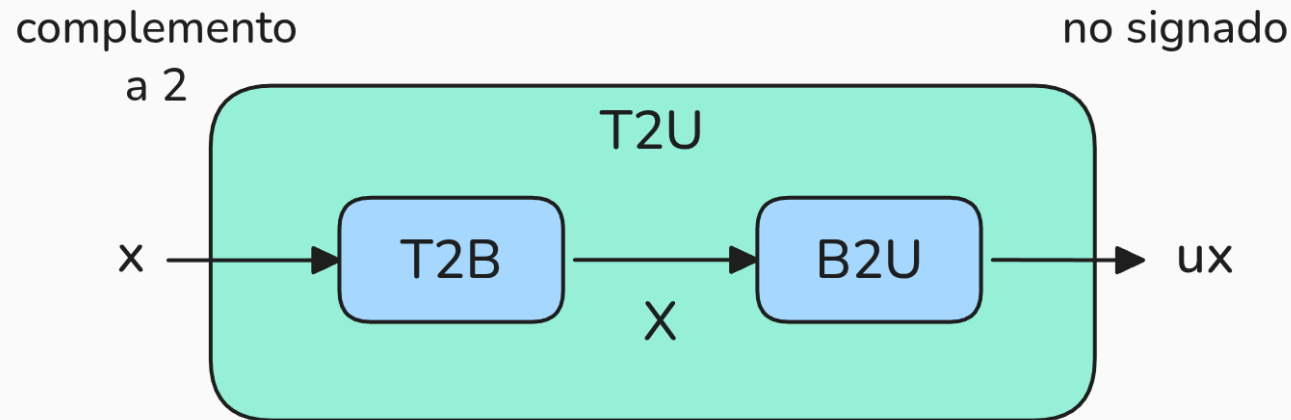
- **⇒ se pueden invertir los mapeos**

- $U2B(X) = B2U^{-1}(X)$: patrón de bits para un entero sin signo
- $T2B(X) = B2T^{-1}(X)$: patrón de bits para un entero en complemento a dos

Reinterpretación de signado a no signado



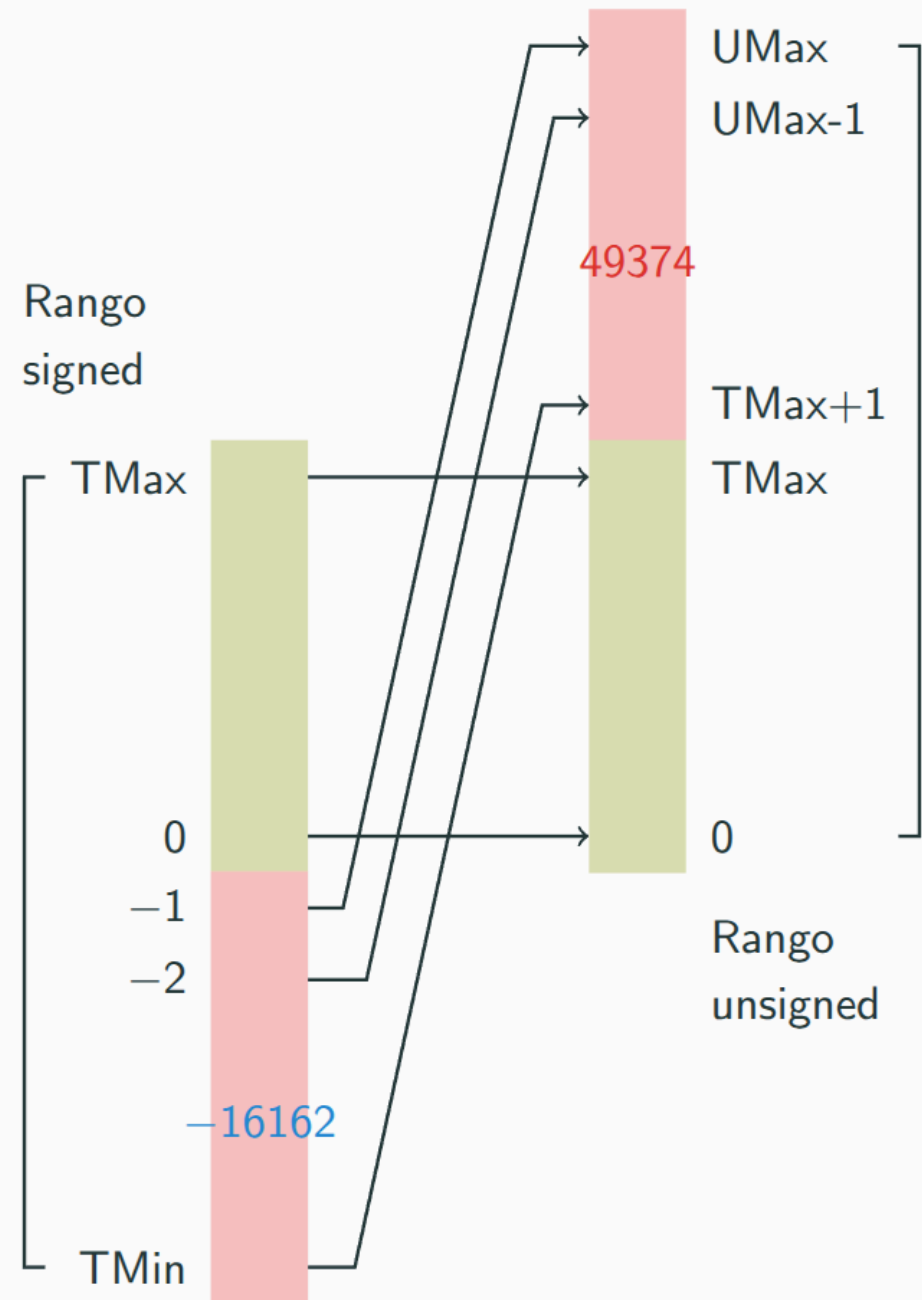
Reinterpretación de signado a no signado



No cambia el patrón de bits: mantener los bits y reinterpretar

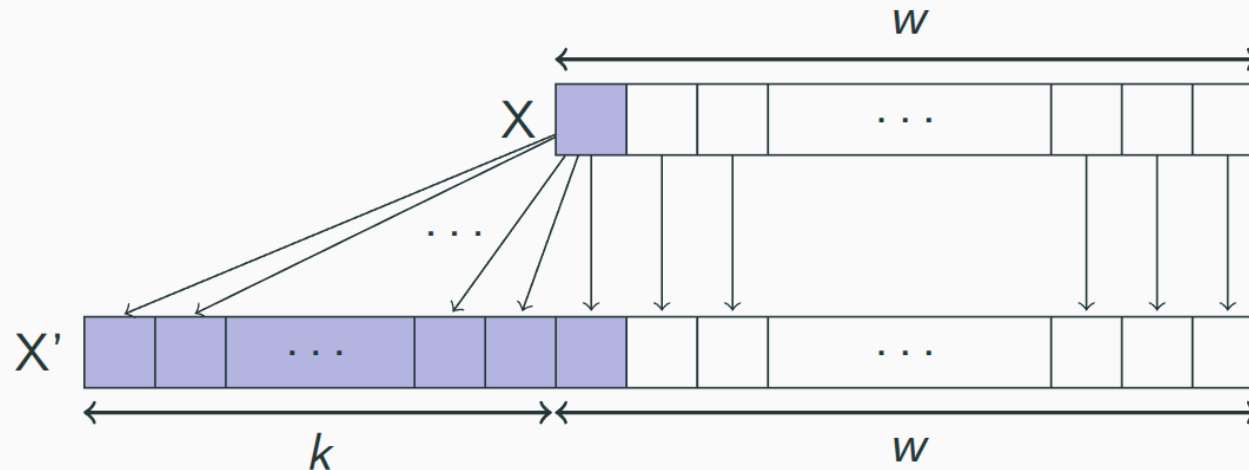
Relación entre signado y no signado

Peso (2^i)	-16162	49374
1	0	0
2	1	1
4	1	1
8	1	1
16	1	1
32	0	0
64	1	1
128	1	1
256	0	0
512	0	0
1024	0	0
2048	0	0
4096	0	0
8192	0	0
16384	1	1
± 32768	-32768	32768
Suma:	-16162	49374



- **Queremos:**
 - Dado un entero *signado* de w -bits
 - Convertirlo a un entero de $w + k$ -bits con el mismo valor
- **Regla:**
 - Propagar el bit de signo a los nuevos k bits (MSB)

$$X' = \underbrace{x_{w-1}, \dots, x_{w-1}}_{k \text{ copias del MSB}}, \underbrace{x_{w-1}, x_{w-2}, \dots, x_1, x_0}_{\text{entero original}}$$



Extensión de signo: ejemplo

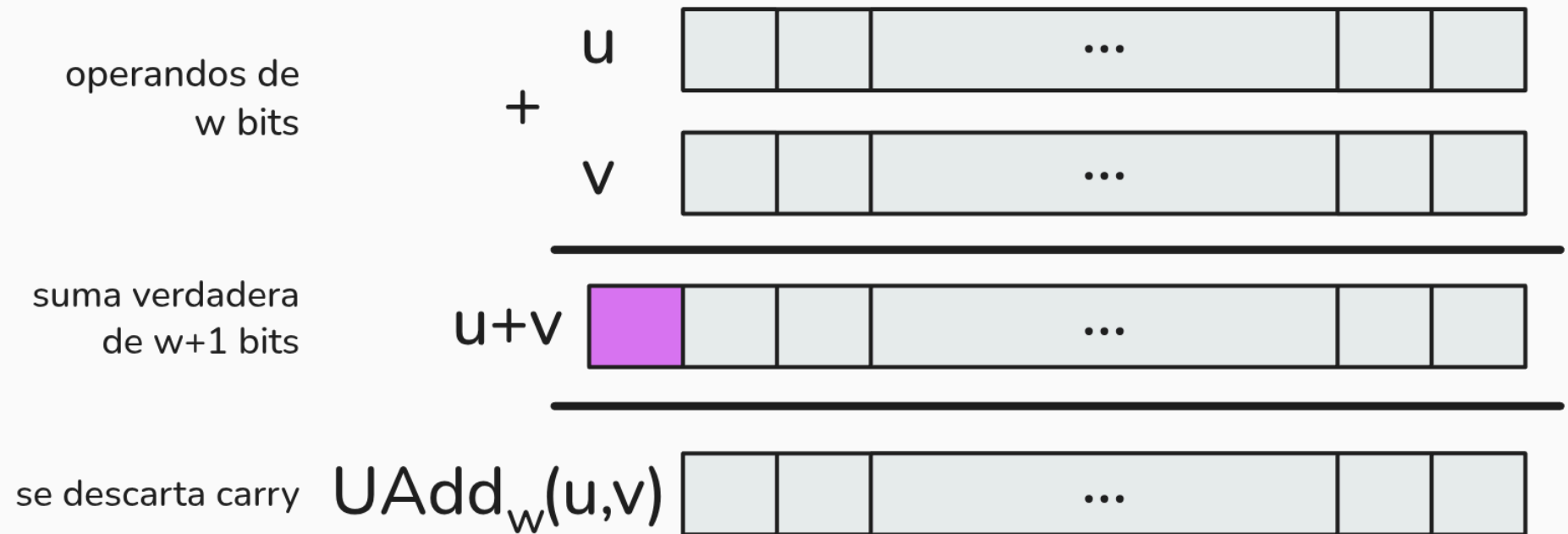
	Decimal	Hex						Binario	
x	16162	3F 22						00111111	00100010
ix	16162	00	00	3F	22	00000000	00000000	00111111	00100010
y	-16162	C0 DE						11000000	11011110
iy	-16162	FF	FF	C0	DE	11111111	11111111	11000000	11011110

Se aumenta la cantidad de bits en la representación: expansión ó *up casting*.

Aritmética



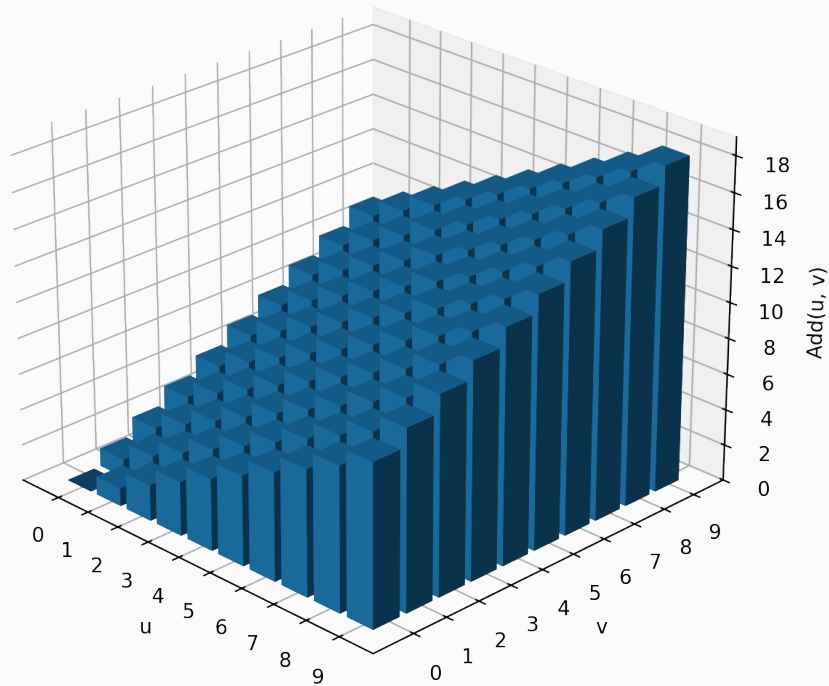
Suma de enteros no signados



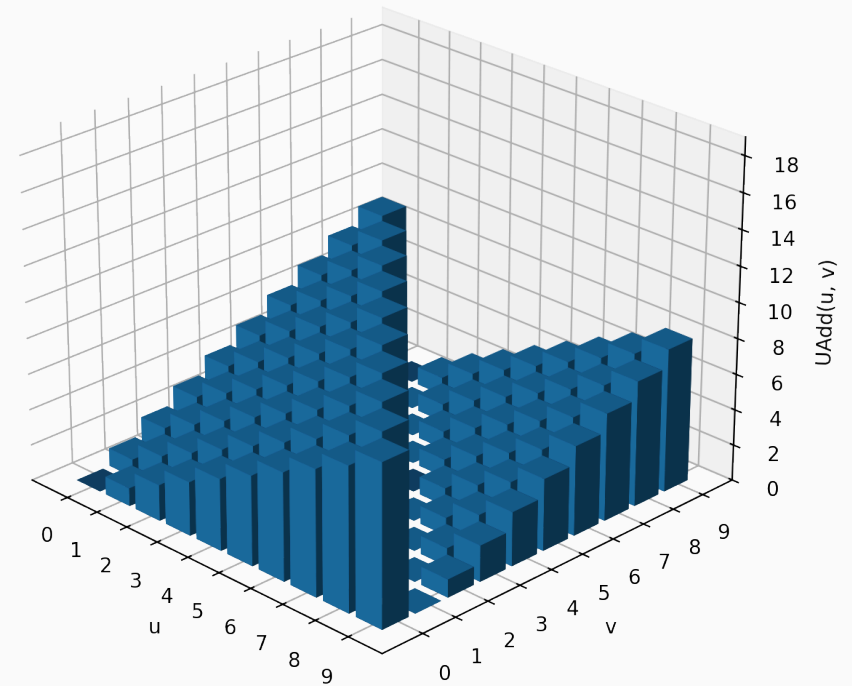
$$\text{UAdd}_w(u, v) = u + v \bmod 2^w$$

Suma de enteros no signados

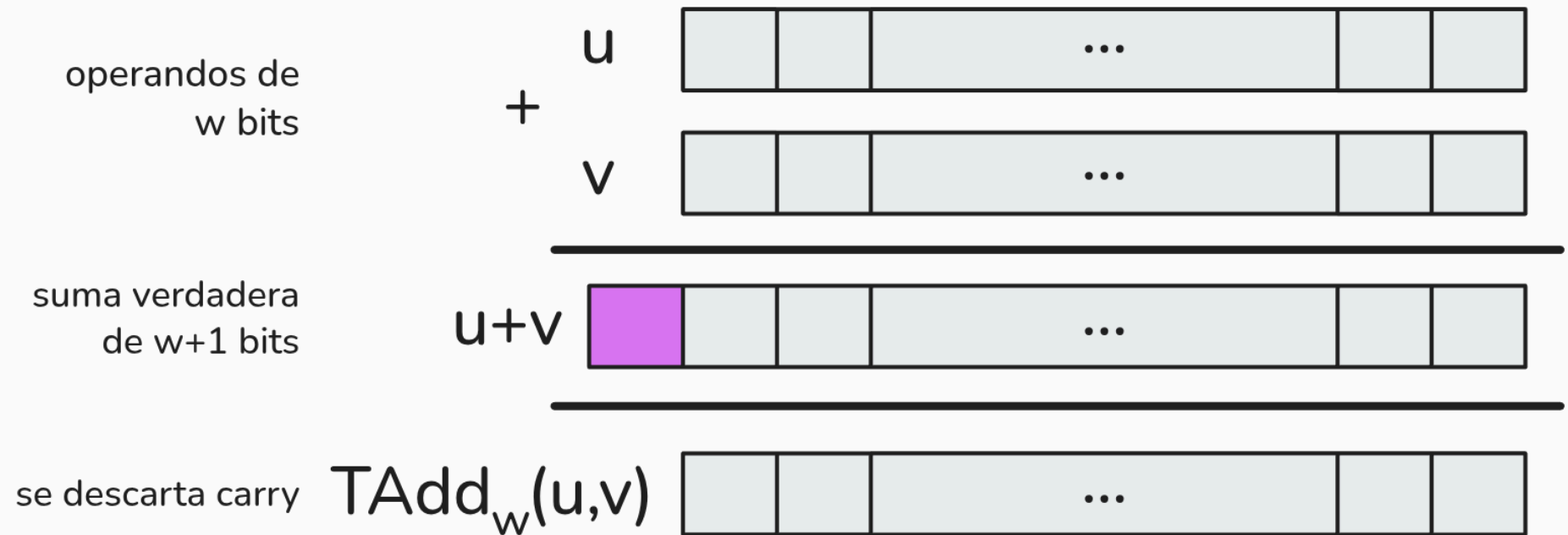
Suma teórica de enteros



Suma de enteros de largo fijo



Suma de enteros en complemento a dos



- **A nivel binario: comportamiento idéntico a la suma unsigned**

Ejemplo con $w = 4 \Rightarrow$

- Requiere $w + 1$ bits
- Descarta el MSB
- El resultado es en CA2
- *wraps around*
 - Si la suma $\geq 2^{w-1}$: se vuelve negativa (no más de una vez)
 - Si la suma $< -2^{w-1}$: se vuelve positiva (no más de una vez)

u		v		suma		TAdd ₄ (u, v)		
bin	dec	bin	dec	bin	dec	msb	4-bit	dec
0111	7	0111	7	01110	14	1	1110	−2
0111	7	0110	6	01101	13	1	1101	−3
0111	7	0101	5	01100	12	1	1100	−4
0110	6	0100	4	01010	10	1	1010	−6
0011	3	0110	6	01001	9	1	1001	−7
0110	6	0010	2	01000	8	1	1000	−8
0100	4	0011	3	00111	7	0	0111	7
0000	0	0110	6	00110	6	0	0110	6
0101	5	0000	0	00101	5	0	0101	5
1101	−3	0110	6	00011	3	0	0011	3
1111	−1	0010	2	00001	1	0	0001	1
1001	−7	0111	7	00000	0	0	0000	0
0011	3	1100	−4	11111	−1	1	1111	−1
1000	−8	0101	5	11101	−3	1	1101	−3
0100	4	1000	−8	11100	−4	1	1100	−4
1001	−7	0010	2	11011	−5	1	1011	−5
1000	−8	0001	1	11001	−7	1	1001	−7
1010	−6	1110	−2	11000	−8	1	1000	−8
1000	−8	1111	−1	10111	−9	0	0111	7
1010	−6	1100	−4	10110	−10	0	0110	6
1001	−7	1100	−4	10101	−11	0	0101	5
1000	−8	1100	−4	10100	−12	0	0100	4
1000	−8	1001	−7	10001	−15	0	0001	1
1000	−8	1000	−8	10000	−16	0	0000	0

¿Cómo detectamos en una suma **sin signo** si el resultado es válido? → **CARRY**

¿Cómo detectamos en una suma **sin signo** si el resultado es válido? -> **CARRY**

¿Cómo detectamos en una suma **con signo** si el resultado es válido? -> **OVERFLOW**

¿Cómo detectamos en una suma **sin signo** si el resultado es válido? → **CARRY**

¿Cómo detectamos en una suma **con signo** si el resultado es válido? → **OVERFLOW**

Ahora...¿qué demonios es el *overflow*?

Sobre el *Overflow* y el *Carry*...

Algunos ejemplos para pensar **en C2** (complemento a 2)...

Sobre el *Overflow* y el *Carry*...

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{rcccc} & 1 & 1 & 0 & 1 \\ & 0 & 1 & 0 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 & 0 \end{array}$$

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} \overset{1}{0} \overset{1}{1} \overset{0}{0} \overset{1}{1} \\ + 1 \ 1 \ 0 \ 1 \\ \hline \overset{1}{1} \ 0 \ 0 \ 1 \ 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{rcccc} & 1 & 1 & 0 & 1 \\ & 0 & 1 & 0 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 & 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

Sobre el *Overflow* y el *Carry*...

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} \overset{1}{0} \overset{1}{1} \overset{0}{0} \overset{1}{1} \\ + 1 \ 1 \ 0 \ 1 \\ \hline \overset{1}{1} \ 0 \ 0 \ 1 \ 0 \end{array}$$

$$\begin{array}{r} \overset{1}{1} \overset{1}{0} \overset{1}{1} \overset{1}{1} \\ + 1 \ 1 \ 0 \ 1 \\ \hline \overset{1}{1} \ 1 \ 0 \ 0 \ 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

Sobre el *Overflow* y el *Carry*...

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{rcccc} & 1 & 1 & 0 & 1 \\ & 0 & 1 & 0 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 & 0 \end{array}$$

$$\begin{array}{rcccc} & 1 & 1 & 1 & 1 \\ & 1 & 0 & 1 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 1 & 0 & 0 & 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{rcccc} & 1 & 1 & 0 & 1 \\ & 0 & 1 & 0 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 & 0 \end{array}$$

$$\begin{array}{rcccc} & 1 & 1 & 1 & 1 \\ & 1 & 0 & 1 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 1 & 0 & 0 & 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$-5 - 4 = (-5) + (-4) = -9$$

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{rcccc} & 1 & 1 & 0 & 1 \\ & 0 & 1 & 0 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 & 0 \end{array}$$

$$\begin{array}{rcccc} & 1 & 1 & 1 & 1 \\ & 1 & 0 & 1 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 1 & 0 & 0 & 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$-5 - 4 = (-5) + (-4) = -9$$

$$\begin{array}{rcccc} & 1 & 0 & 0 & 0 \\ & 1 & 0 & 1 & 1 \\ + & 1 & 1 & 0 & 0 \\ \hline 1 & 0 & 1 & 1 & 1 \end{array}$$

Sobre el *Overflow* y el *Carry*...

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{rcccc} & 1 & 1 & 0 & 1 \\ & 0 & 1 & 0 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 & 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$\begin{array}{rcccc} & 1 & 1 & 1 & 1 \\ & 1 & 0 & 1 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 1 & 0 & 0 & 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$-5 - 4 = (-5) + (-4) = -9$$

$$\begin{array}{rcccc} & 1 & 0 & 0 & 0 \\ & 1 & 0 & 1 & 1 \\ + & 1 & 1 & 0 & 0 \\ \hline 1 & 0 & 1 & 1 & 1 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 0 \oplus 1 = 1$$

OVERFLOW!

Sobre el *Overflow* y el *Carry*...

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{rcccc} & 1 & 1 & 0 & 1 \\ & 0 & 1 & 0 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 0 & 1 & 0 \end{array}$$

$$\begin{array}{rcccc} & 1 & 1 & 1 & 1 \\ & 1 & 0 & 1 & 1 \\ + & 1 & 1 & 0 & 1 \\ \hline 1 & 1 & 0 & 0 & 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$-5 - 4 = (-5) + (-4) = -9$$

$$5 + 4 = 9$$

$$\begin{array}{rcccc} & 1 & 0 & 0 & 0 \\ & 1 & 0 & 1 & 1 \\ + & 1 & 1 & 0 & 0 \\ \hline 1 & 0 & 1 & 1 & 1 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 0 \oplus 1 = 1$$

OVERFLOW!

Sobre el *Overflow* y el *Carry*...

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 1 1 0 1 \\ 0 1 0 1 \\ + 1 1 0 1 \\ \hline 1 0 0 1 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 1 1 1 1 \\ 1 0 1 1 \\ + 1 1 0 1 \\ \hline 1 1 0 0 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$-5 - 4 = (-5) + (-4) = -9$$

$$\begin{array}{r} 1 0 0 0 \\ 1 0 1 1 \\ + 1 1 0 0 \\ \hline 1 0 1 1 1 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 0 \oplus 1 = 1$$

OVERFLOW!

$$5 + 4 = 9$$

$$\begin{array}{r} 0 1 0 0 \\ 0 1 0 1 \\ + 0 1 0 0 \\ \hline 0 1 0 0 1 \end{array}$$

Sobre el *Overflow* y el *Carry*...

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 1 1 0 1 \\ 0 1 0 1 \\ + 1 1 0 1 \\ \hline 1 0 0 1 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 1 1 1 1 \\ 1 0 1 1 \\ + 1 1 0 1 \\ \hline 1 1 0 0 0 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 1 = 0$$

OK!

$$-5 - 4 = (-5) + (-4) = -9$$

$$\begin{array}{r} 1 0 0 0 \\ 1 0 1 1 \\ + 1 1 0 0 \\ \hline 1 0 1 1 1 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 0 \oplus 1 = 1$$

OVERFLOW!

$$5 + 4 = 9$$

$$\begin{array}{r} 0 1 0 0 \\ 0 1 0 1 \\ + 0 1 0 0 \\ \hline 0 1 0 0 1 \end{array}$$

$$C_{\text{in}} \oplus C_{\text{out}} = 1 \oplus 0 = 1$$

OVERFLOW!

Podemos detectar Overflow haciendo un XOR entre el carry que entra al MSB y el carry que sale del MSB

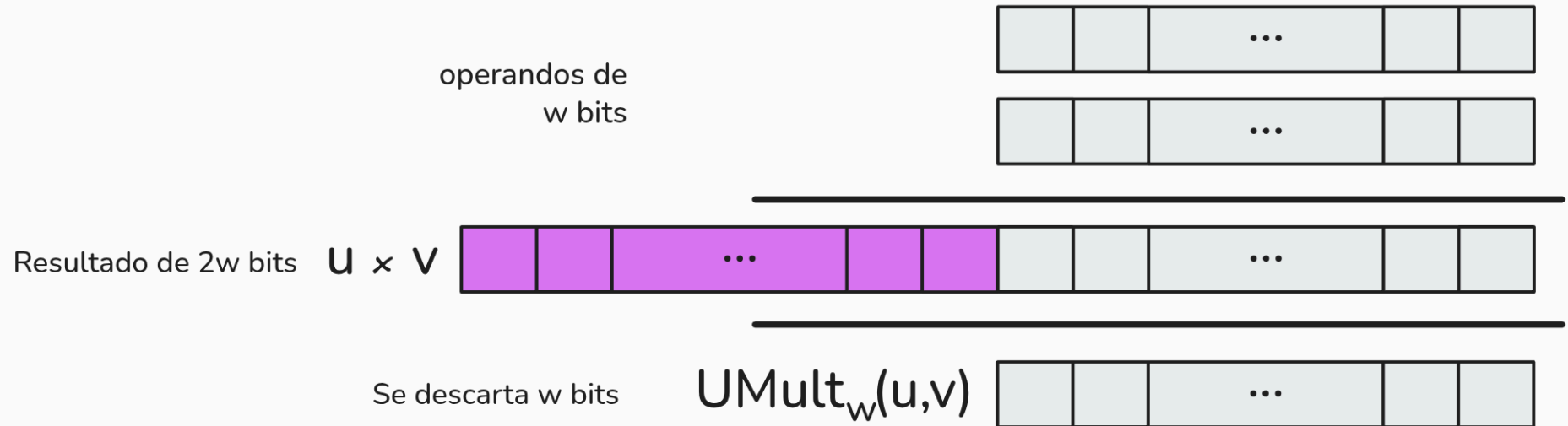
Podemos detectar Overflow haciendo un XOR entre el carry que entra al MSB y el carry que sale del MSB

Hay otra forma de detectar el overflow:

- si el bit de signo es igual en ambos operandos (ambos positivos o negativos) el resultado de la suma debería preservar el signo
- si el bit de signo es distinto en los operandos, el overflow no puede ocurrir

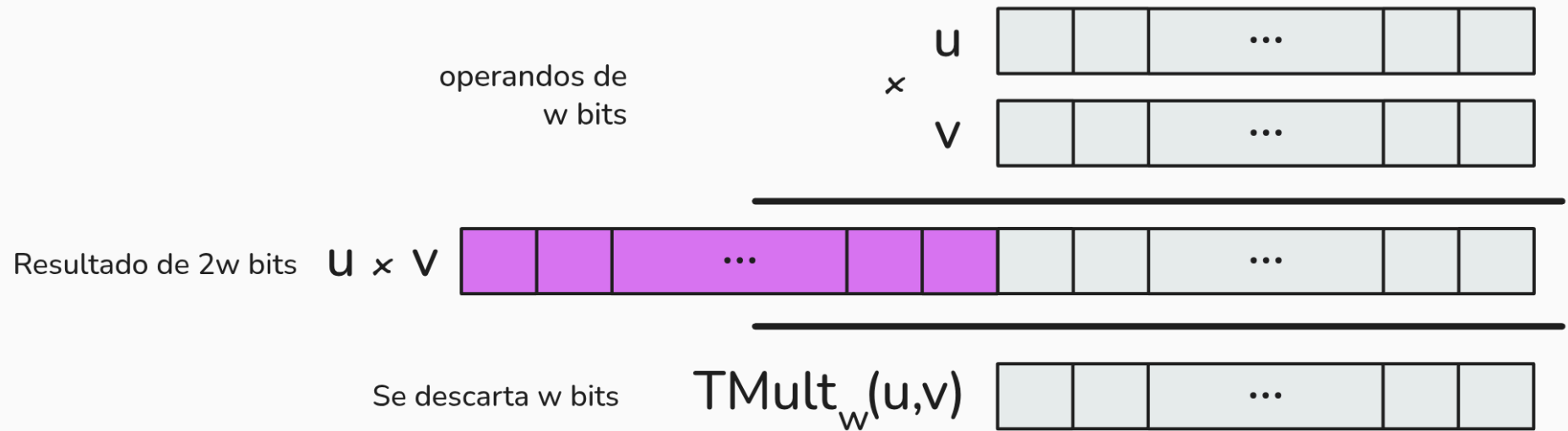
$$\text{overflow} \Leftrightarrow ((a_{n-1} = b_{n-1}) \wedge (a_{n-1} \neq s_{n-1}))$$

Multiplicación de enteros



- **Multiplicación estándar**
 - ignora los w bits superiores
- **Implementa aritmética modular**
 - como ocurre con la suma:

$$\text{UMult}_w(u, v) = u \cdot v \bmod 2^w$$



- **Multiplicación estándar**

- ignora los w bits superiores
- los bits dentro de los w superiores pueden diferir
- los bits inferiores son iguales a $\text{UMult}_w(u, v)$

- Representación numérica

$$N_2 = \sum_{i=0}^{w-1} b_i \cdot 2^i$$

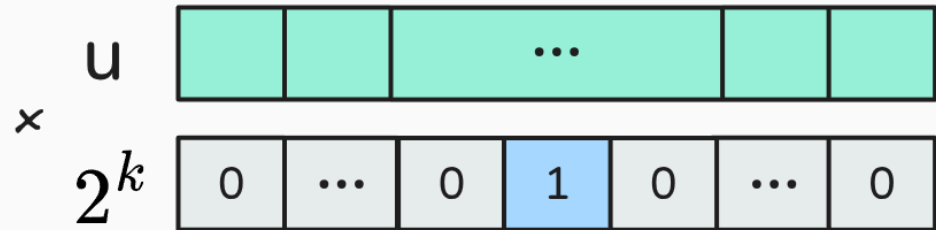
- Multiplicar por 2^k
 - requiere k bits más

$$\begin{aligned} N_2 \cdot 2^k &= \left(\sum_{i=0}^{w-1} b_i \cdot 2^i \right) \cdot 2^k \\ &= \sum_{i=0}^{w-1} b_i \cdot 2^{i+k} \\ &= \sum_{i=k}^{w+k-1} b_i \cdot 2^i + \sum_{j=0}^{k-1} 0 \cdot 2^j \end{aligned}$$

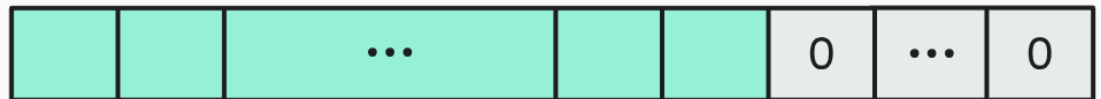
- ignora los k bits superiores

Multiplicar por potencias de 2: shift

operandos de
w bits



Resultado de w+k bits $u \times 2^k$



Se descarta k bits



Operación

$u \cdot 2^k$ se puede hacer con desplazamientos: $u \ll k$

Operación

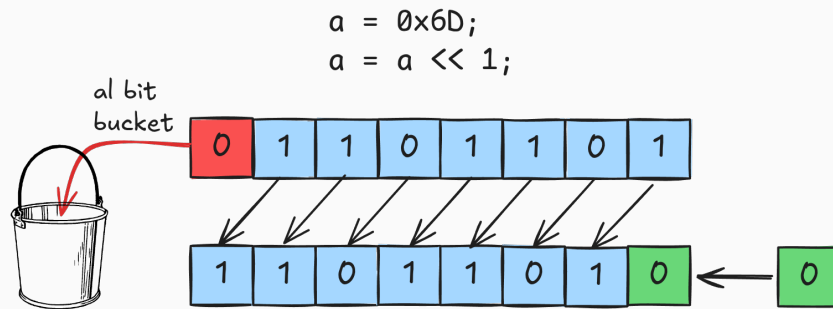
$\lfloor u/2^k \rfloor$ se puede hacer con desplazamientos: $u \gg k$

Ejemplos

	Teórica	Real	Hex	Binario
x	16162	16162	3F 22	00111111 00100010
x >> 1	8081.0	8081	1F 91	00011111 10010001
x >> 2	4040.50	4040	0F C8	00001111 11001000
x >> 8	63.133	63	00 3F	00000000 00111111
y	-16162	-16162	C0 DE	11000000 11011110
y >> 1	-8081.0	-8081	E0 6F	11100000 01101111
y >> 2	-4040.50	-4041	F0 37	11110000 00110111
y >> 8	-63.133	-64	FF C0	11111111 11000000

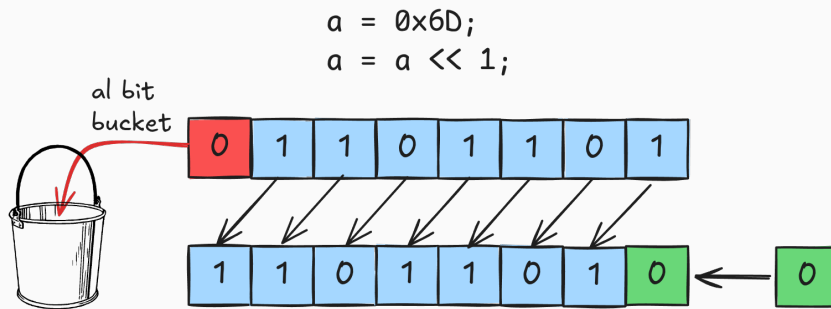
Desplazamiento lógico

- A izquierda

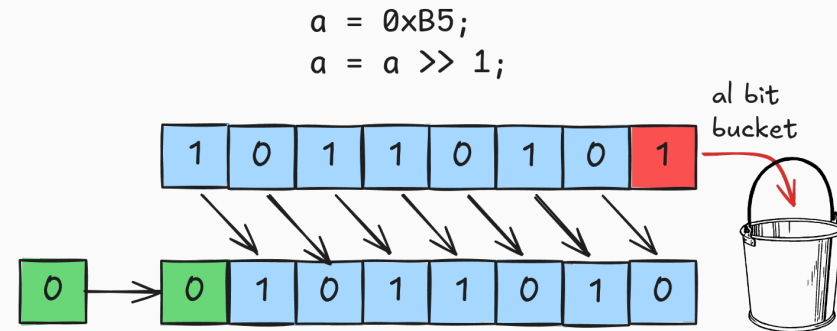


Desplazamiento lógico

- A izquierda



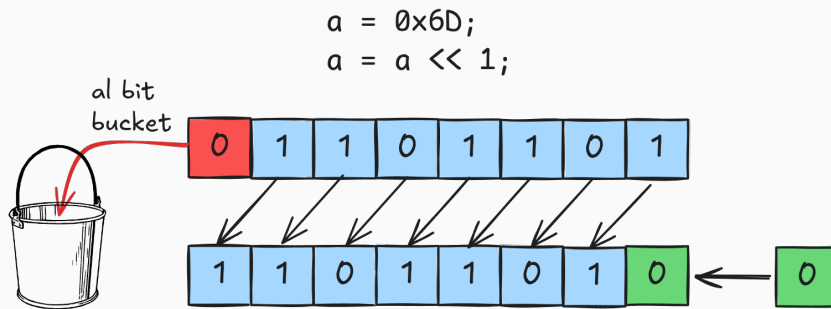
- A derecha



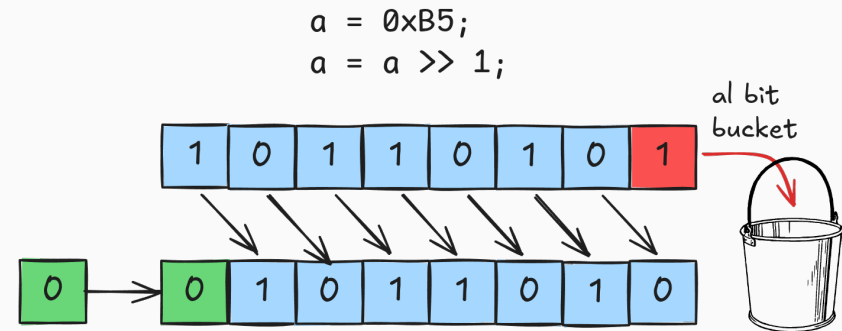
Desplazamiento aritmético

Desplazamiento lógico

- A izquierda

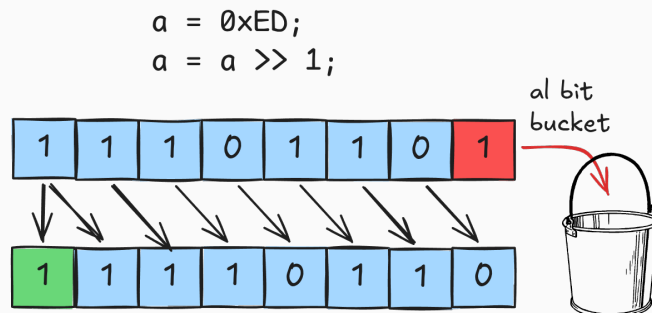


- A derecha



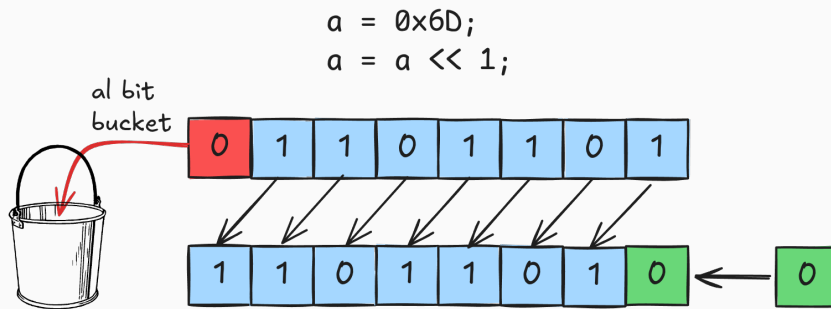
Desplazamiento aritmético

- MSB: 1

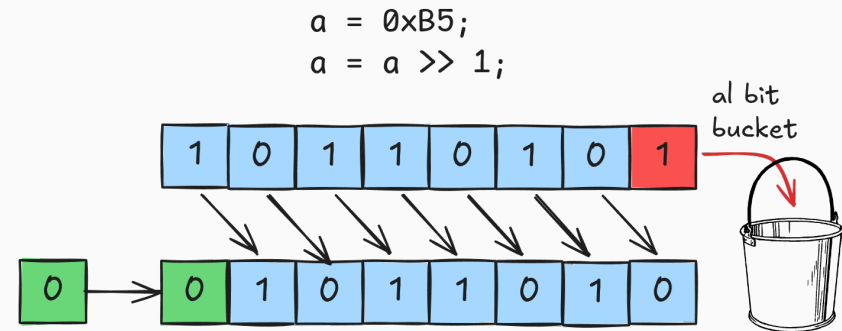


Desplazamiento lógico

- A izquierda

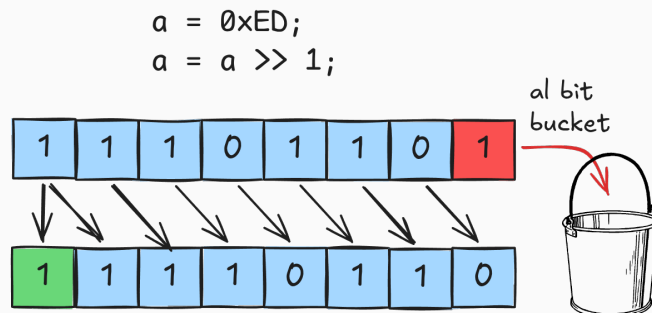


- A derecha

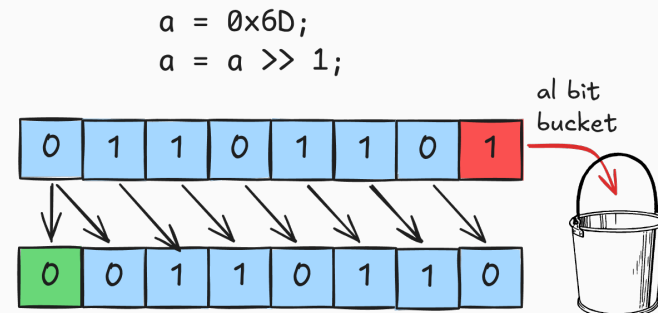


Desplazamiento aritmético

- MSB: 1



- MSB: 0



Apéndice



En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011		
-2			
8			
-8			

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	
-2			
8			
-8			

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	0011
-2			
8			
-8			

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	0011
-2	No representable		
8			
-8			

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	0011
-2	No representable	1010	
8			
-8			

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	0011
-2	No representable	1010	1110
8			
-8			

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	0011
-2	No representable	1010	1110
8	1000		
-8			

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	0011
-2	No representable	1010	1110
8	1000	No representable	
-8			

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	0011
-2	No representable	1010	1110
8	1000	No representable	No representable
-8			

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	0011
-2	No representable	1010	1110
8	1000	No representable	No representable
-8	No representable		

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	0011
-2	No representable	1010	1110
8	1000	No representable	No representable
-8	No representable	No representable	

En base 2, datos de 4 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0011	0011	0011
-2	No representable	1010	1110
8	1000	No representable	No representable
-8	No representable	No representable	1000

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011		
-2			
8			
-8			

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	
-2			
8			
-8			

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2			
8			
-8			

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable		
8			
-8			

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable	1000 0010	
8			
-8			

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable	1000 0010	1111 1110
8			
-8			

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable	1000 0010	1111 1110
8	0000 1000		
-8			

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable	1000 0010	1111 1110
8	0000 1000	0000 1000	
-8			

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable	1000 0010	1111 1110
8	0000 1000	0000 1000	0000 1000
-8			

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable	1000 0010	1111 1110
8	0000 1000	0000 1000	0000 1000
-8	No representable		

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable	1000 0010	1111 1110
8	0000 1000	0000 1000	0000 1000
-8	No representable	1000 1000	

En base 2, datos de 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable	1000 0010	1111 1110
8	0000 1000	0000 1000	0000 1000
-8	No representable	1000 1000	1111 1000

Similitudes entre 4 y 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable	1000 0010	1111 1110
8	0000 1000	0000 1000	0000 1000
-8	No representable	1000 1000	1111 1000

Similitudes entre 4 y 8 bits

	Sin signo	Signo + Magnitud	Complemento a 2
3	0000 0011	0000 0011	0000 0011
-2	No representable	1000 0010	1111 1110
8	0000 1000	0000 1000	0000 1000
-8	No representable	1000 1000	1111 1000

Extendiendo la cantidad de bits de precisión:

- Signo + Magnitud: se extiende con 0's, pero el bit más significativo se mantiene indicando el signo.
- Complemento a 2: se extiende con el valor del bit más significativo.

Sigla	Se lee	Qué hace
$B2U_w(X)$	Bits to Unsigned	Interpreta X como entero sin signo
$B2T_w(X)$	Bits to Two's complement	Interpreta X como entero en CA2
$U2B(x)$	Unsigned to Bits	Inversa de B2U: bits del entero sin signo x
$T2B(x)$	Two's complement to Bits	Inversa de B2T: bits del entero CA2 x
$U2T(x)$	Unsigned to Two's complement	Reinterpreta los mismos bits: sin signo $\rightarrow$ CA2
$T2U(x)$	Two's complement to Unsigned	Reinterpreta los mismos bits: CA2 $\rightarrow$ sin signo
UMin, UMax	Unsigned Min/Max	Mínimo y máximo representable sin signo
TMin, TMax	Two's complement Min/Max	Mínimo y máximo representable en CA2
$C2(b)$	Complemento a 2 de b	$2^n - b$: inverso aditivo de b en n bits
$UAdd_w(u, v)$	Unsigned Add	$u + v \bmod 2^w$
$TAdd_w(u, v)$	Two's complement Add	Suma en CA2 – mismo circuito que UAdd
$UMult_w(u, v)$	Unsigned Mult	$u \cdot v \bmod 2^w$
$TMult_w(u, v)$	Two's complement Mult	Multiplicación en CA2 – iguales w bits bajos que UMult

El subíndice w indica la cantidad de bits de la representación. Las “2B” (U2B, T2B) son inversas de las “B2” (B2U, B2T); U2T y T2U reinterpretan el mismo patrón de bits sin modificarlo.